

Policy Optimization for Financial Decision-Making

Comparative Analysis of Deep Learning vs Reinforcement Learning Approaches

Author: Shreyansh Singh

Submitted for: Shodh AI - Machine Learning Engineer Position

Date: October 2025

Executive Summary

This project explores two different approaches to automated loan approval: a deep learning classifier that predicts default probability, and a reinforcement learning agent that directly optimizes for profitability. Using LendingClub's historical loan data (2007-2018), I trained both models and compared their decisions on 105,000 test loans.

The key finding is that the RL agent earns \$54 more per loan on average than the DL model, which adds up to \$5.62 million in additional profit. What's interesting is that the RL agent achieves this by being *more selective* - it approves 17% fewer loans than the DL model, but the loans it does approve have much better quality (83.7% repayment rate vs 78.5% baseline).

The models disagree on 17,845 loans (17% of the test set), and analyzing these disagreements reveals an important insight: **optimizing for predictive accuracy is not the same as optimizing for business outcomes**. The DL model, trained to maximize classification accuracy, tends to approve almost everything. The RL agent, trained to maximize expected profit, correctly identifies and rejects loans with negative expected value.

1. Introduction and Problem Framing

Business Context

Lending institutions face a fundamental decision: which loan applications should be approved? Traditional credit scoring models focus on predicting default risk, but this doesn't directly translate to profitability. A high-risk loan might still be profitable if the interest rate compensates for the risk, while a low-risk loan with minimal interest might not be worth the opportunity cost.

Research Question

Can a reinforcement learning agent that directly optimizes for financial return outperform a traditional deep learning classifier? More specifically, how do the decisions differ when we optimize for classification accuracy versus expected profit?

Approach

I built two models:

1. **Deep Learning Model:** A neural network trained to predict $P(\text{default})$ using binary cross-entropy loss. The decision rule is: approve if $P(\text{default}) < 0.5$.
2. **RL Agent:** A policy that learns to maximize expected profit per loan, where reward = profit from interest (if paid) or -principal (if default).

Both models use the same features and are evaluated on the same test set, making the comparison fair and interpretable.

2. Dataset and Preprocessing

Data Source

The LendingClub dataset contains 2.26 million loan applications from 2007-2018 with 151 features. After filtering for completed loans (excluding current or late loans), 1.37 million records remained.

Sampling Strategy

Rather than using all 1.37M records, I sampled 700K loans using stratified random sampling to maintain the class distribution (78.5% fully paid, 21.5% default). This sample size was chosen to ensure statistical robustness while being computationally practical - it represents 51% of all completed loans, providing a substantial and representative dataset. The larger sample also helps capture more rare patterns in the data, especially for high-grade and low-grade loans.

Feature Engineering and Selection

The original 151 features contained several issues:

Data Leakage: Many features like `total_pymnt`, `last_pymnt_amnt`, and `recoveries` are only known *after* the loan outcome. Including these would artificially inflate model performance. I carefully removed all 18 post-outcome features.

Missing Values: Features like `mths_since_last_delinq` had >50% missing values. After analysis, I either dropped these or imputed with sensible defaults.

Redundant Features: The dataset had both `grade` and `sub_grade`, which are highly correlated. Similarly, I created a single `fico_score` feature from `fico_range_low` and `fico_range_high` by taking their average.

Final Feature Set: 21 features covering borrower characteristics (income, employment, FICO), loan terms (amount, interest rate, term), and credit history (DTI, revolving utilization, total accounts).

Train/Validation/Test Split

- Training: 70% (489,981 loans)
- Validation: 15% (105,018 loans) - for early stopping
- Test: 15% (105,000 loans) - held out for final evaluation

All splits maintained the 78.5/21.5 class distribution through stratification.

Key EDA Insights

One clear pattern emerged: default rate increases monotonically with loan grade. Grade A loans default only 7% of the time, while Grade G loans default 51% of the time. This makes sense - higher grades mean lower assessed risk, which correlates with better outcomes. However, Grade G loans also have much higher interest rates (24% vs 7% for Grade A), which becomes important when thinking about profitability rather than just default prediction.

3. Deep Learning Model

Architecture

I built a fairly standard feed-forward neural network in PyTorch:

- Input layer: 21 features
- Hidden layer 1: 128 neurons + ReLU + Dropout(0.3)
- Hidden layer 2: 64 neurons + ReLU + Dropout(0.3)
- Hidden layer 3: 32 neurons + ReLU + Dropout(0.2)
- Output layer: 1 neuron + Sigmoid (outputs probability)

After testing a few architectures, this one gave the best validation performance. The dropout layers help prevent overfitting, which was an issue with larger networks.

Training Details

- Loss: Binary Cross-Entropy
- Optimizer: Adam with learning rate 0.001
- Batch size: 512 (optimal for M3 MPS)
- Max epochs: 30 with early stopping (patience=7)
- Device: Apple Silicon MPS (GPU acceleration)

Training took 3.4 minutes and stopped at epoch 26 when validation AUC plateaued at 0.7169.

Performance Results

Test Set Metrics:

- AUC-ROC: **0.7175** (strong discrimination between paid/default)
- Accuracy: 78.93%
- Precision: 59.72%
- Recall: 5.94% (very low!)
- F1-Score: 0.1080

Analysis of Results

The AUC of 0.7175 is actually quite good - it matches the best published results on Kaggle for models that don't use data leakage. Many notebooks report AUC > 0.94, but they include post-outcome features like outstanding principal and last payment amount, which obviously inflate performance.

However, the recall of 5.94% is problematic. The model only catches 1,339 out of 22,559 actual defaults (5.9%), missing 94.1% of them. Looking at the confusion matrix, the model predicts "default" for only 1,339 loans total, when there are actually 22,559 defaults in the test set.

Why is recall so low? Two reasons:

1. **Class imbalance:** With 78.5% paid loans, the model learns that predicting "paid" is usually correct. Binary cross-entropy doesn't account for class weights, so the model becomes very conservative about predicting default.
2. **Threshold sensitivity:** Using 0.5 as the decision threshold is arbitrary. I computed the optimal threshold using Youden's index and found it should be 0.1997. At this lower threshold, recall jumps to 69.1% with F1-score of 0.452, though precision drops to 34%.

The key takeaway is that this model is extremely liberal in approvals - it only denies 2,242 out of 105,000 loans (2.1% denial rate, 97.9% approval rate). While this might seem like it's "helping more people," it's actually approving many loans that will likely default.

4. Reinforcement Learning Agent

RL Problem Formulation

Instead of predicting default probability, I framed loan approval as a sequential decision problem:

- **State:** The 21 loan application features
- **Action:** {0 = Deny, 1 = Approve}
- **Reward:**
 - Deny → \$0
 - Approve + Paid → $\text{loan_amount} \times \text{int_rate} \times (\text{term}/12)$ (profit from interest)
 - Approve + Default → $-\text{loan_amount}$ (loss of principal)

The goal is to learn a policy that maximizes expected reward per loan.

Why This Formulation Makes Sense

Traditional credit scoring focuses on $P(\text{default})$, but profitability depends on both risk and return. Consider two scenarios:

Scenario A:

- Loan: \$5,000 at 7% interest, 36 months (Grade A)
- Default prob: 10%
- Expected value: $0.9 \times \$1,050 - 0.1 \times \$5,000 = +\$445$ (POSITIVE)

Scenario B:

- Loan: \$20,000 at 16% interest, 36 months (Grade E)
- Default prob: 40%
- Expected value: $0.6 \times \$9,600 - 0.4 \times \$20,000 = -\$2,240$ (NEGATIVE)

The key insight is that higher interest rates don't always compensate for higher risk, especially when loan amounts are large.

Training Approach

Since I'm working with offline historical data (all loans already approved and completed), I can't use traditional RL algorithms that require environment interaction. Instead, I used supervised learning to train a policy:

Algorithm: Gradient Boosting Regressor (200 trees) **Target:** Expected reward if loan is approved (calculated from actual outcomes) **Training data:** All 489,981 training loans with their actual outcomes

The idea is simple: learn to predict the reward we would get from approving each loan, then approve only loans with positive expected reward.

Performance Results

Test Set Metrics:

- Expected Policy Value: **\$5,021.98 per loan**
- Total Profit: \$527.31M (on 105,000 test loans)
- Approval Rate: 80.9% (approved 84,921 loans)
- Loan Quality: 83.7% repayment rate
- Training Time: 3.4 minutes

Comparison vs Baselines:

- vs "Approve All": +3.3% better profit
- vs "Random": +107.8% better profit
- vs Deep Learning: +\$53.56 per loan (+1.1%)

What Drives RL Decisions?

Feature importance analysis shows the top factors:

1. **sub_grade** (40%): Most important - captures combined credit risk
2. **int_rate** (10%): Higher rates increase potential profit
3. **dti** (6%): Debt-to-income ratio signals repayment ability
4. **term** (6%): Loan duration affects total interest collected
5. **annual_inc** (4%): Income level affects ability to repay

What's notable is that the RL agent considers *both* risk indicators (sub_grade, dti) and return indicators (int_rate, term) together, whereas the DL model only sees risk.

5. Comparative Analysis

Decision Agreement

The models agree on 83.0% of loans (87,147 loans):

- **Both Approve:** 84,913 loans (80.9%) → 83.7% paid, \$527.0M profit
- **Both Deny:** 2,234 loans (2.1%) → 40.2% would have paid (correctly conservative)

The models disagree on 17.0% of loans (17,853 loans):

- **RL Approve, DL Deny:** 8 loans (0.01%) → 50/50 paid/default split
- **RL Deny, DL Approve:** 17,845 loans (17.0%) → This is the interesting case!

Deep Dive: The 17,845 Disagreement Cases

These are loans that the DL model approves but the RL agent denies. What makes them different?

Characteristics (vs agreed approvals):

Feature	Disagreement Loans	Agreed Approvals	Difference
Loan Amount	\$15,825	\$15,663	+1% larger
Interest Rate	15.8%	15.7%	+0.1pp higher
DTI Ratio	Higher	Lower	Worse debt position
Sub-grade	Lower quality	Higher quality	More D-G grades

Actual Outcomes:

- 10,488 would have paid (58.8%)
- 7,357 would have defaulted (41.2%)

The default rate of 41.2% is much higher than the 16.3% rate for agreed approvals. Even though the majority (58.8%) would pay, the expected value is negative due to the large loss when defaults occur.

Why Does DL Approve Them?

Remember that the DL model has 5.94% recall - it almost never predicts default. With a 0.5 threshold, the model must be >50% confident of default to deny a loan. For most of these 17,845 loans, the DL model outputs probabilities around 0.3-0.5 (high risk but not >50%), so it approves them.

Why Does RL Deny Them?

Let's calculate the expected value for a typical disagreement loan:

Loan Amount: \$15,825
Interest Rate: 15.8%
Term: 36 months
Default Probability: 41.2%

Expected Value:
= $P(\text{paid}) \times \text{profit} - P(\text{default}) \times \text{loss}$
= $0.588 \times (\$15,825 \times 0.158 \times 3) - 0.412 \times \$15,825$
= $0.588 \times \$7,501 - 0.412 \times \$15,825$
= $\$4,410 - \$6,520$
= $-\$2,110$

The expected value is negative. Even though there's a 59% chance of repayment, the potential loss from the 41% default probability outweighs the profit. The RL agent, directly optimizing for expected value, correctly identifies these as unprofitable and denies them.

Financial Impact

If the DL model's decisions were used:

- Total profit: \$521.68M

If the RL agent's decisions were used:

- Total profit: \$527.31M

Difference: +\$5.62M (+1.1% improvement)

This comes entirely from avoiding the 17,845 unprofitable loans. Even though 59% of them would have paid, the losses from the 41% that default eliminate any profit and then some.

Grade-wise Analysis

The disagreements are concentrated in lower credit grades:

Grade % of Grade where Models Disagree RL Approval Rate

A	0.3%	99.9%
B	2.8%	97.1%
C	8.5%	89.2%
D	20.4%	76.8%
E	34.1%	62.5%
F	46.7%	50.2%
G	71.8%	27.1%

For high-quality loans (Grade A-B), both models agree to approve. For low-quality loans (Grade F-G), the RL agent becomes very selective, denying most applications that the DL model approves.

This makes sense: lower grades have higher default rates and need higher interest rates to be profitable. The RL agent has learned that many high-risk loans don't have sufficient interest to compensate for the risk.

6. Limitations and Future Work

Current Limitations

1. **Simplified Reward Function:** The current reward assumes full repayment or full default. In reality, borrowers might make partial payments before defaulting, or pay off early. A more sophisticated reward would model the payment schedule over time and include recovery rates.
2. **Static Historical Data:** The models are trained on 2007-2018 data. Economic conditions, borrower demographics, and lending standards have all changed since then. The models might not perform as well on current applications.
3. **No Fairness Constraints:** The models might discriminate against certain demographic groups. While I removed explicit demographic features, proxies like ZIP code (encoded in address) could still introduce bias.
4. **Single Algorithm:** I only tested gradient boosting for the RL agent. Other offline RL algorithms like Conservative Q-Learning (CQL) or Implicit Q-Learning (IQL) might perform better.
5. **Threshold Dependence:** The DL model's performance is highly sensitive to the classification threshold. With the optimal threshold (0.1997), the performance might be closer to the RL agent.

Future Improvements

Short-term:

- Test the DL model at its optimal threshold and compare fairly
- Try ensemble methods (multiple RL agents voting)
- Incorporate loan term duration more explicitly into the reward function
- Add temporal validation (train on 2007-2015, validate on 2016-2018)

Advanced research directions:

- **Causal inference:** Use doubly robust estimation or instrumental variables to estimate causal effects of approval decisions
- **Fairness-aware RL:** Add constraints to ensure equal opportunity across demographic groups
- **Contextual bandits:** Deploy in a production system with exploration to continuously learn from new data
- **Multi-objective optimization:** Balance profit, risk, fairness, and customer satisfaction simultaneously
- **Explainable AI:** Use SHAP values or counterfactual explanations to make decisions interpretable for compliance

7. Conclusions

Key Findings

1. **Optimization objective matters:** The DL model optimizes for classification accuracy and approves 97.9% of loans. The RL agent optimizes for profitability and approves 80.9% of loans, achieving \$5.62M higher profit by being more selective.
2. **Accuracy $\neq$ Profitability:** The DL model achieves strong AUC (0.7175), matching state-of-the-art results. However, its liberal approval strategy results in lower profit. Predictive accuracy is necessary but not sufficient for business value.
3. **The disagreement analysis reveals the mechanism:** The RL agent denies 17,845 loans with negative expected value that the DL model approves. These loans have 41.2% default rate vs 16.3% for agreed approvals.
4. **Small per-loan differences scale:** The \$54 per loan advantage seems small, but across thousands of loans it becomes substantial. In a real lending portfolio with millions of loans, this could mean tens of millions in additional profit.

Broader Implications

This work demonstrates that **directly optimizing for the business objective is better than optimizing for a proxy metric**. This principle extends beyond lending:

- **Healthcare**: Optimize for patient outcomes (quality-adjusted life years), not just diagnostic accuracy
- **Marketing**: Optimize for customer lifetime value, not just click-through rate
- **Supply chain**: Optimize for total cost of ownership, not just demand forecast error
- **Content recommendation**: Optimize for long-term engagement, not just immediate clicks

Personal Reflection

When I started this project, I expected the RL agent to be more aggressive - approving risky loans with high interest rates that the DL model would deny. Instead, the opposite happened: the RL agent is more conservative overall, but makes smarter trade-offs between risk and return.

The most interesting finding for me was the 17,845 disagreement cases. These loans have a majority (59%) repayment rate, so intuitively it seems like they should be approved. But when you calculate the expected value accounting for the large losses from the 41% that default, it becomes clear they're unprofitable. This is exactly the kind of nuanced decision that's hard to capture with a binary classification model.

If I were deploying this in production, I would probably use a hybrid approach: use the DL model to generate default probability estimates, then use an RL-style expected value calculation for the final decision. This would combine the interpretability of probabilities with the business-focused optimization of RL.

References

1. LendingClub Dataset (Kaggle): <https://www.kaggle.com/datasets/wordsforthewise/lending-club>
 2. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.
 3. Levine, S., Kumar, A., Tucker, G., & Fu, J. (2020). "Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems." arXiv:2005.01643
 4. PyTorch Documentation: <https://pytorch.org/docs/stable/index.html>
 5. Scikit-learn Documentation: <https://scikit-learn.org/stable/>
-